

LAB 7

SERVER & SERVICES

IIS • Linux Web Service • DNS

Prepared by: Mohamed Sikkanthar Asaf

Eleven tasks covering PC/router/server addressing, standing up IIS and a Linux web service, configuring DNS, and verifying both sites resolve and load correctly by name from the client.

Contents

Overview & Topology

IP Address Plan

Task 1 — PC IP Assignment

Task 2 — Router IP Assignment

Task 3 — Win Server IP Assignment

Task 4 — Enabling IIS Service in Win Server

Task 5 — Verifying the IIS Web Service from Client PC

Task 6 — Linux Server IP Assignment

Task 7 — Enabling Web Service in Linux Server

Task 8 — Verifying the Linux Web Service from Client PC

Task 9 — DNS Server IP Configuration

Task 10 — Configuring DNS Entries in DNS Server

Task 11 — Adding DNS IP in the Client PC

Task 12 — Verifying the DNS from the Client PC

Conclusion

Overview & Topology

One router connects a client PC to a small “datacenter” segment (a Windows Server running IIS and a Linux server running a lightweight web service) and a separate DNS segment. The lab builds the network up in layers: address every device, bring each web service online and confirm it by IP, then configure DNS so the client reaches both sites by name instead of by IP.

Named hosts used throughout this lab:

- Winserver (“Netflix”) — Windows Server running IIS, at 172.16.0.20
- Linux-1 (“Hotstar”) — Linux server running a lightweight web service (thttpd), at 172.16.0.10
- DNS server at 8.8.8.8, resolving www.netflix.com and www.hotstar.com

IP Address Plan

Device	Interface	IP Address	Subnet Mask	Gateway
Client PC (Win)	eth0	50.0.0.10	255.0.0.0	50.0.0.1
Router	e0/0 (→ Datacenter)	172.16.0.1	255.255.0.0	—
Router	e0/1 (→ DNS)	8.0.0.1	255.0.0.0	—
Router	e0/2 (→ PC)	50.0.0.1	255.0.0.0	—
Win Server — IIS (“Netflix”)	eth0	172.16.0.20	255.255.0.0	172.16.0.1
Linux Server — Web (“Hotstar”)	eth0	172.16.0.10	255.255.0.0	172.16.0.1
DNS Server	eth0	8.8.8.8	255.0.0.0	8.0.0.1

TASK 1 PC IP Assignment

Static IP set on the client PC's NIC via TCP/IP Properties.

- IP address: 50.0.0.10
- Subnet mask: 255.0.0.0
- Default gateway: 50.0.0.1

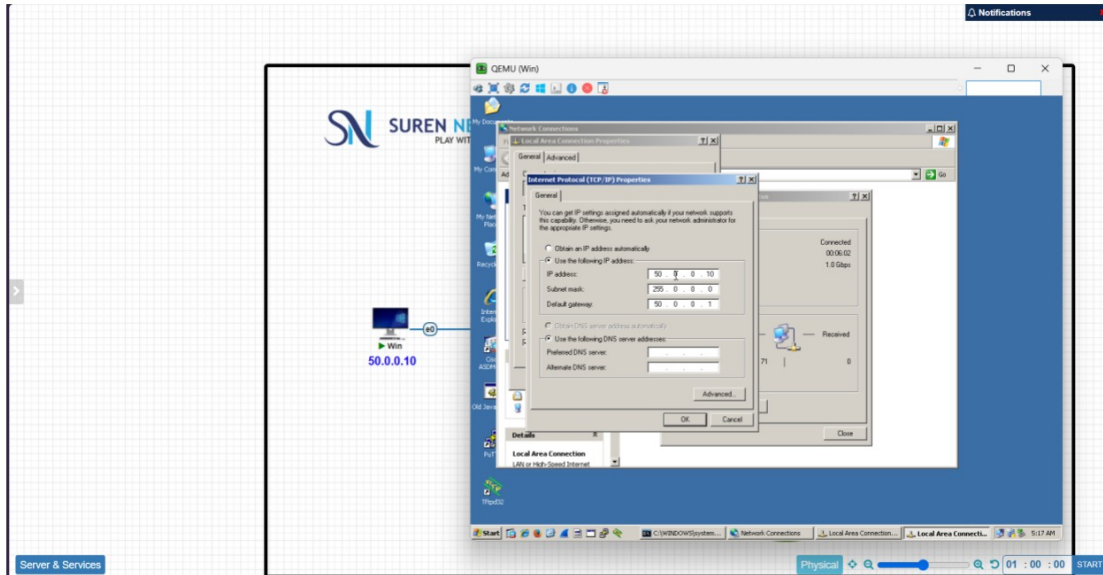


Figure 1 — Client PC TCP/IP configuration

TASK 2 Router IP Assignment

All three router interfaces addressed and brought up from the router CLI:

```
int e0/0 → ip add 172.16.0.1 255.255.0.0 → no sh
int e0/1 → ip add 8.0.0.1 255.0.0.0 → no sh
int e0/2 → ip add 50.0.0.1 255.0.0.0 → no sh
```

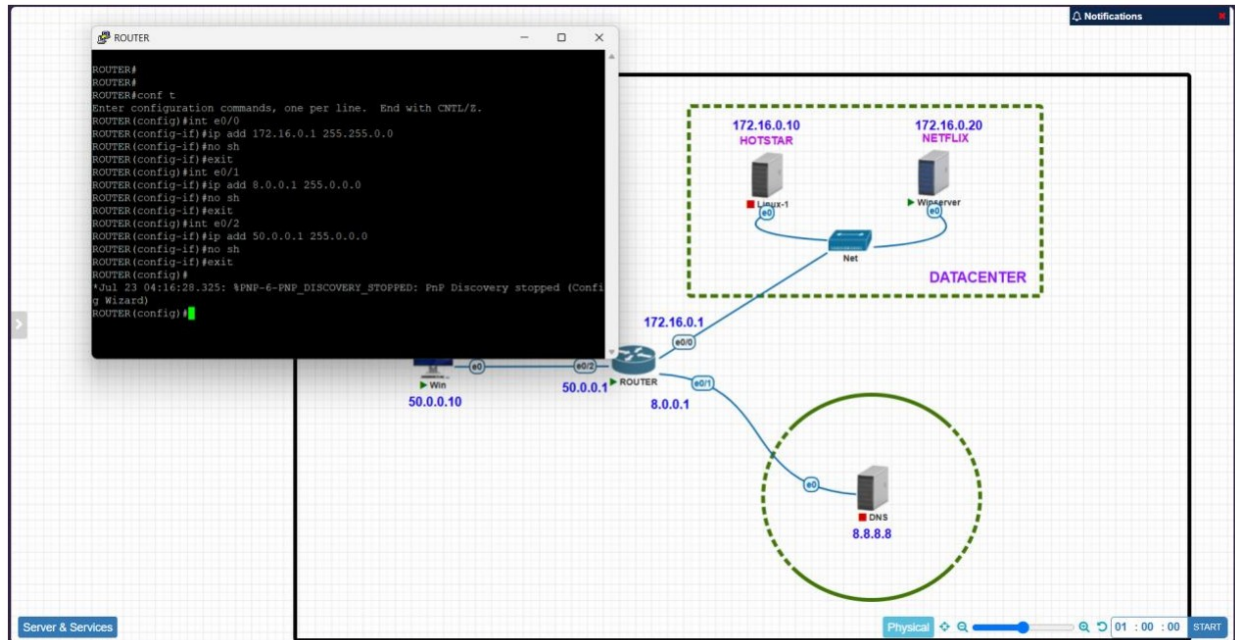


Figure 2 — Router interface configuration and full topology

TASK 3 Win Server IP Assignment

Static IP set on the Windows Server ("Netflix") NIC.

- IP address: 172.16.0.20
- Subnet mask: 255.255.0.0
- Default gateway: 172.16.0.1

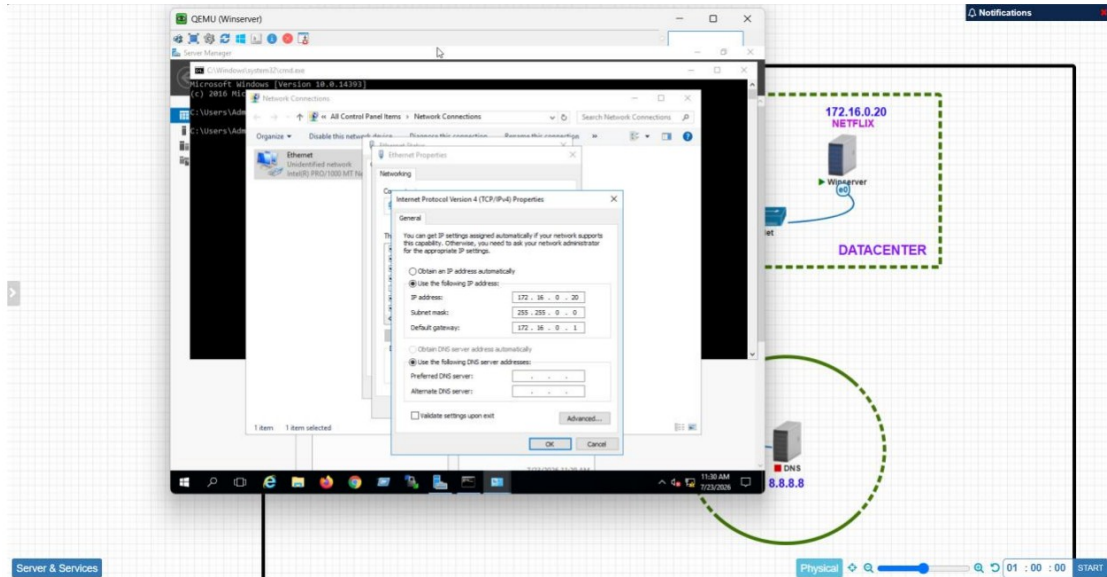


Figure 3 — Win Server TCP/IPv4 configuration

TASK 4 Enabling IIS Service in Win Server

Server Manager → Add Roles and Features Wizard → Server Roles → Web Server (IIS) selected for install.

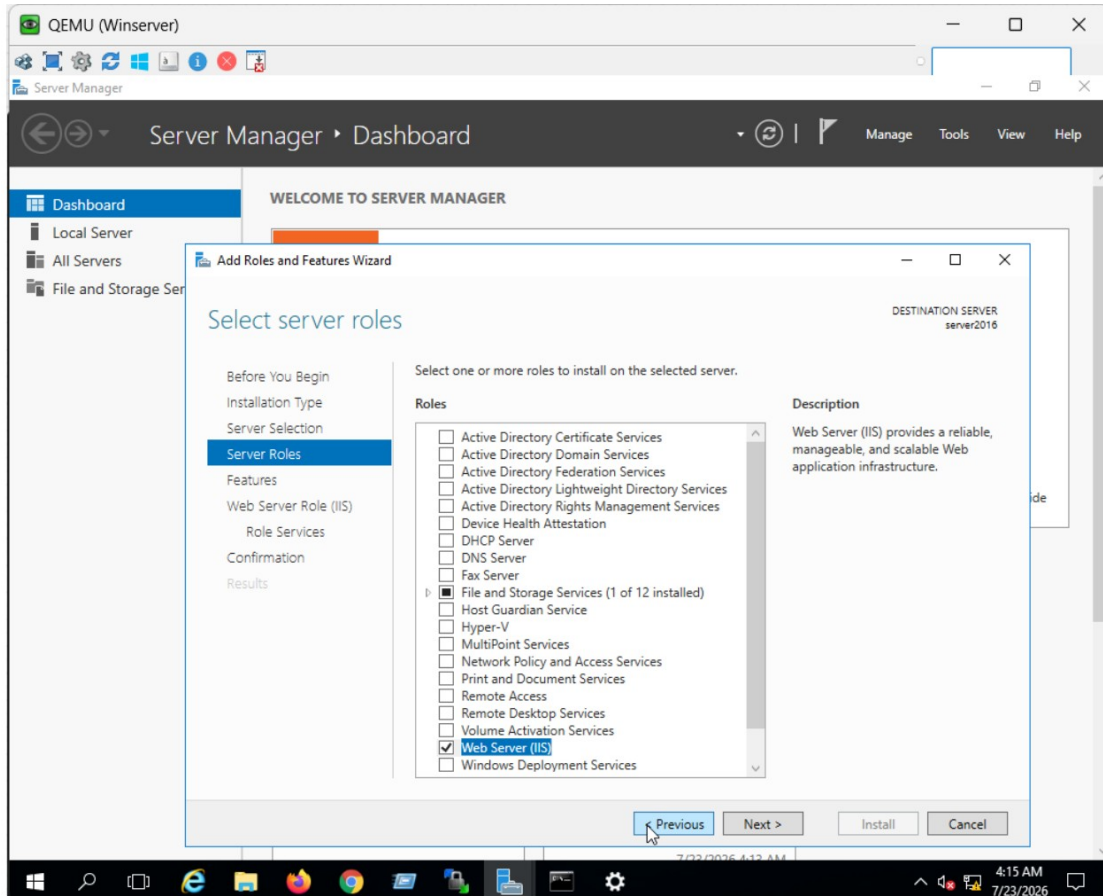


Figure 4 — Add Roles and Features Wizard: Web Server (IIS) role selected

TASK 5 Verifying the IIS Web Service from Client PC

From the client PC, browsing to the Win Server's IP loads the default IIS page — confirming the web service is reachable before DNS is involved.

- URL tested: <http://172.16.0.20>
- Result: default IIS “Internet Information Services” welcome page loads

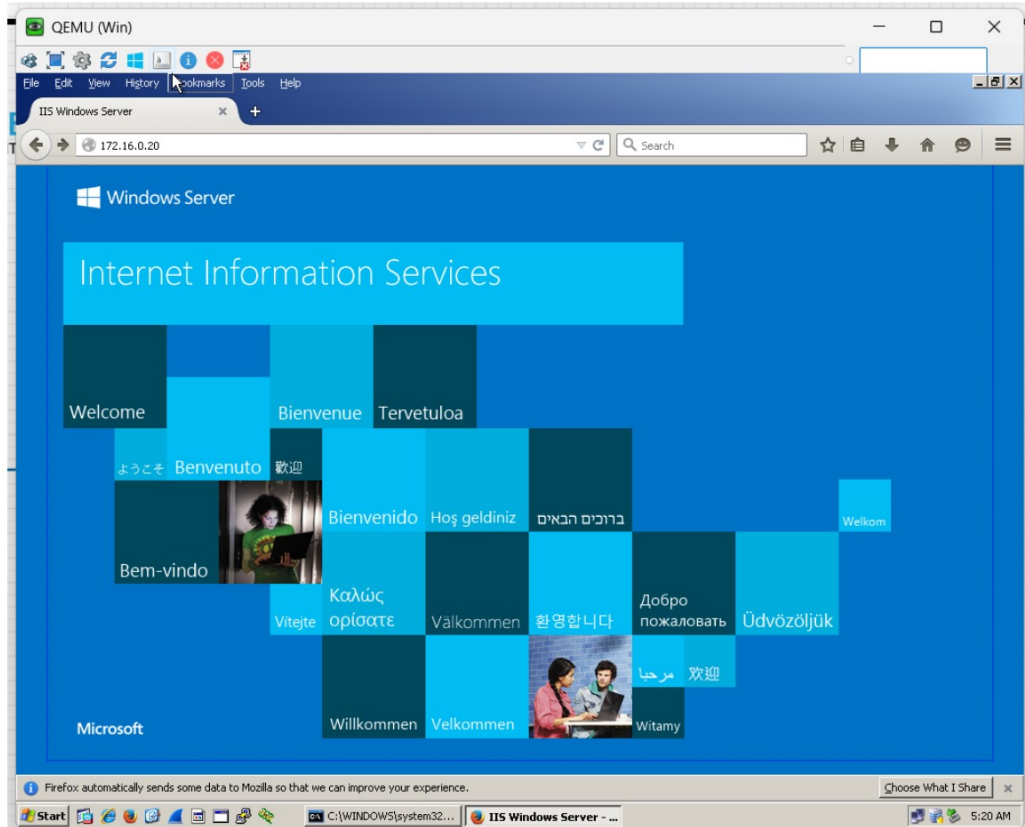


Figure 5 — IIS default page loading by IP in the client browser

TASK 6 Linux Server IP Assignment

Linux-1 (“Hotstar”) boots with a scripted interface config; ifconfig confirms the applied address.

```
eth0: inet addr:172.16.0.10 Bcast:172.16.255.255 Mask:255.255.0.0
```

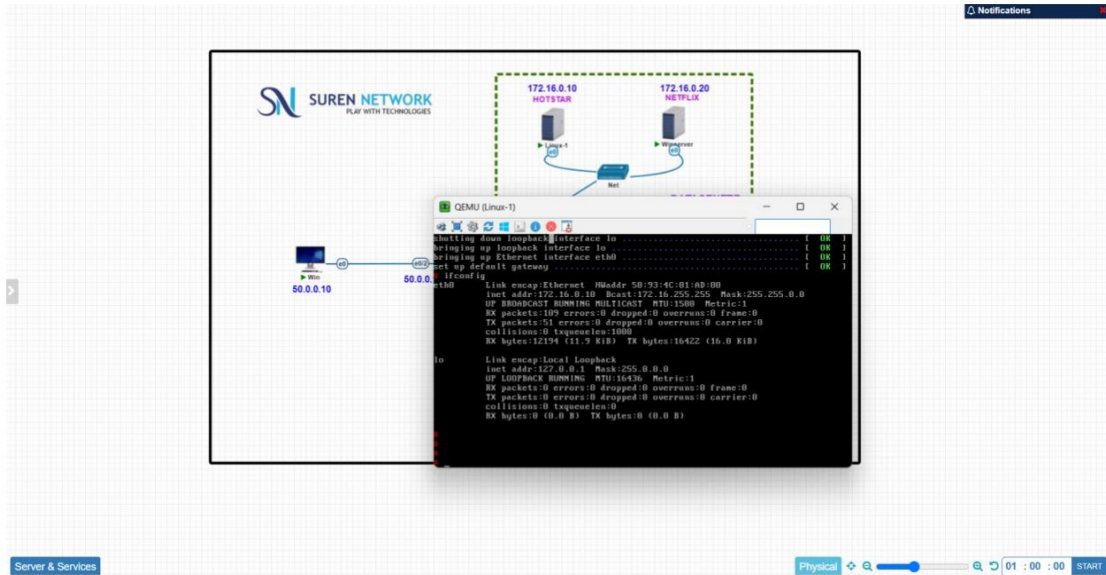



Figure 6 — Linux server boot sequence and ifconfig output

TASK 7 Enabling Web Service in Linux Server

A lightweight web server (thttpd) serves a custom index page from /home/thttpd/index.html.

- Page content: "Welcome to Suren Network — This is the Linux Web Server, Apache is running on it. Thankyou"
- Serving from: /home/thttpd/index.html

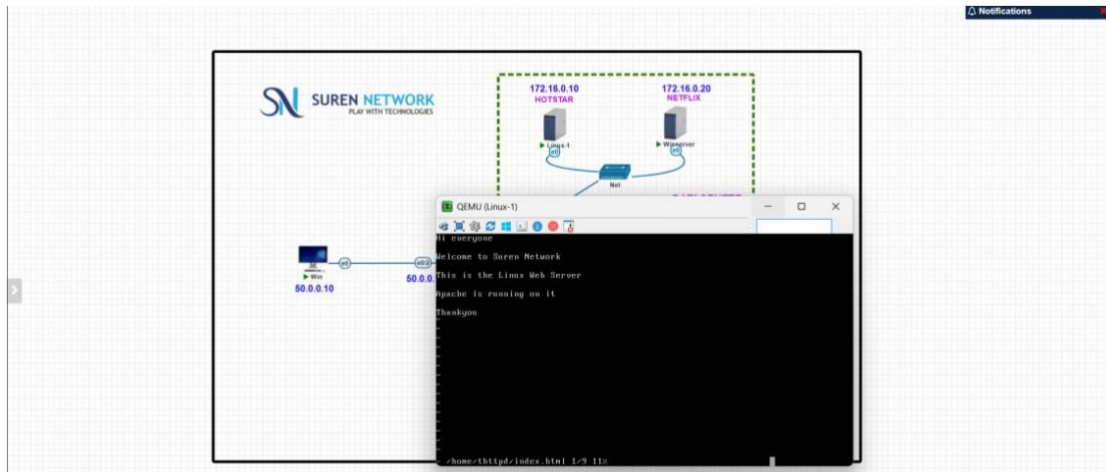


Figure 7 — Linux web service running with custom index page

TASK 8 Verifying the Linux Web Service from Client PC

Same by-IP check as Task 5, this time against the Linux server.

- URL tested: http://172.16.0.10
- Result: custom Linux/thttpd page loads correctly

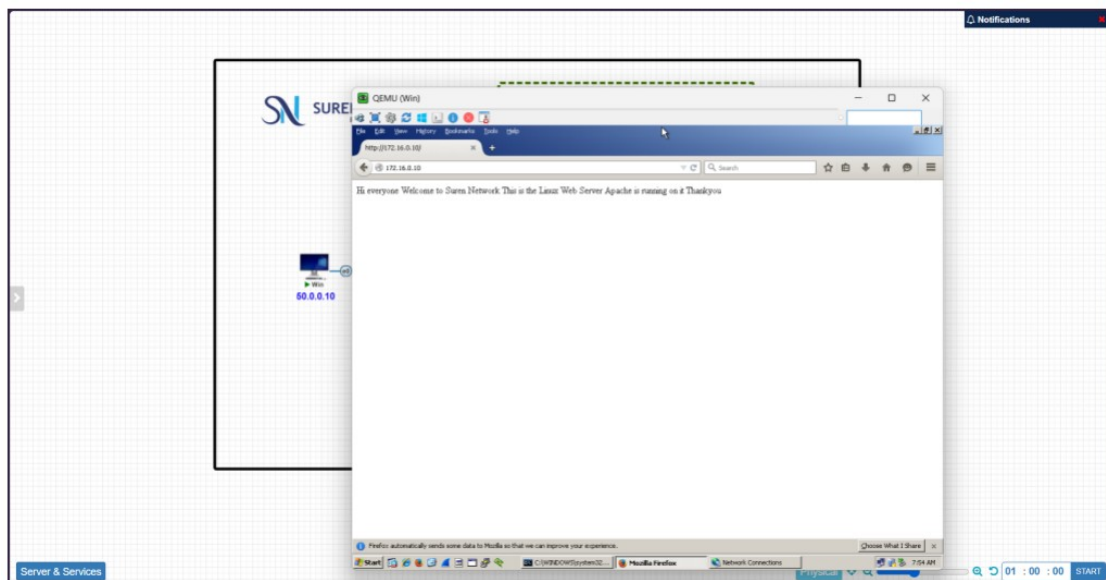


Figure 8 — Linux web page loading by IP in the client browser

TASK 9 DNS Server IP Configuration

Static IP set on the DNS server's NIC.

- IP address: 8.8.8.8
- Subnet mask: 255.0.0.0
- Default gateway: 8.0.0.1

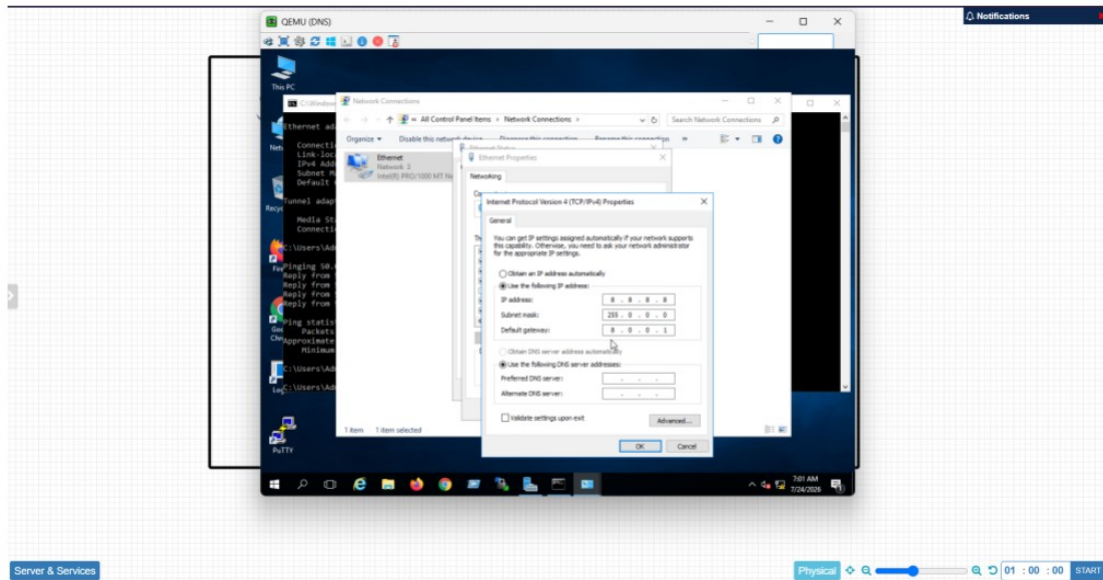


Figure 9 — DNS server TCP/IPv4 configuration

TASK 10 Configuring DNS Entries in DNS Server

DNS Manager → Forward Lookup Zones → com → new host records created under the hotstar and netflix zones.

- www.netflix.com → 172.16.0.20 (Win Server / IIS)
- www.hotstar.com → 172.16.0.10 (Linux Server / tthttpd)

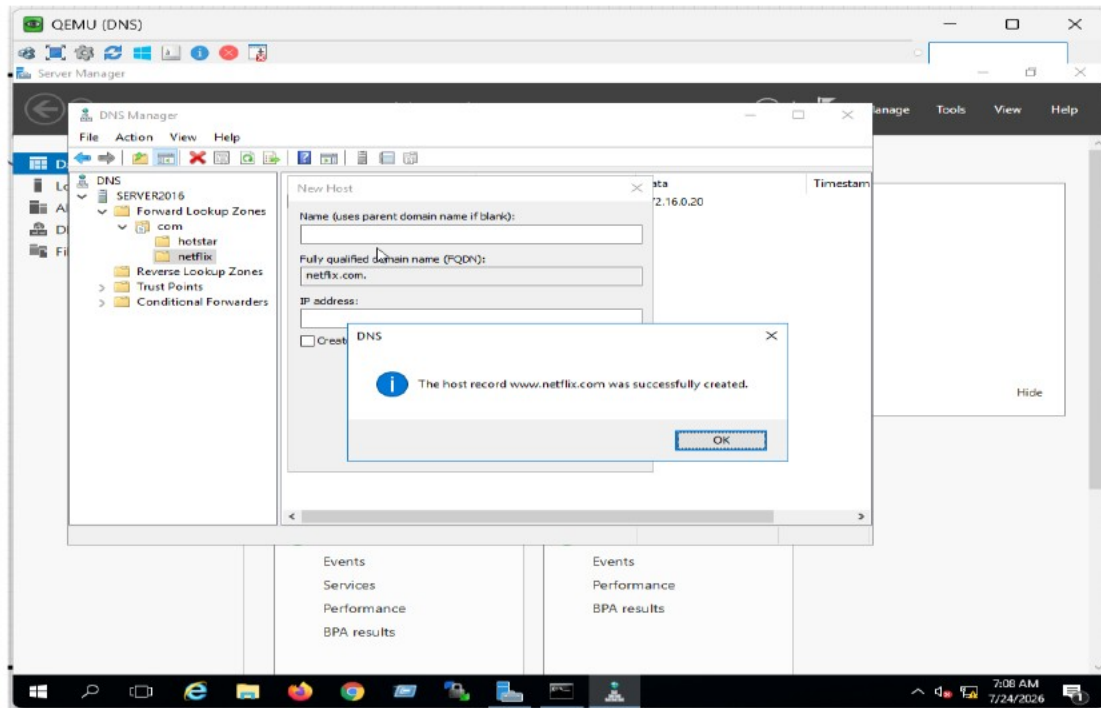


Figure 10 — DNS Manager: host record for `www.netflix.com` created

TASK 11 Adding DNS IP in the Client PC

Client PC's TCP/IP properties updated to use the lab DNS server instead of no DNS server.

- IP address: 50.0.0.10 • Subnet mask: 255.0.0.0 • Gateway: 50.0.0.1
- Preferred DNS server: 8.8.8.8

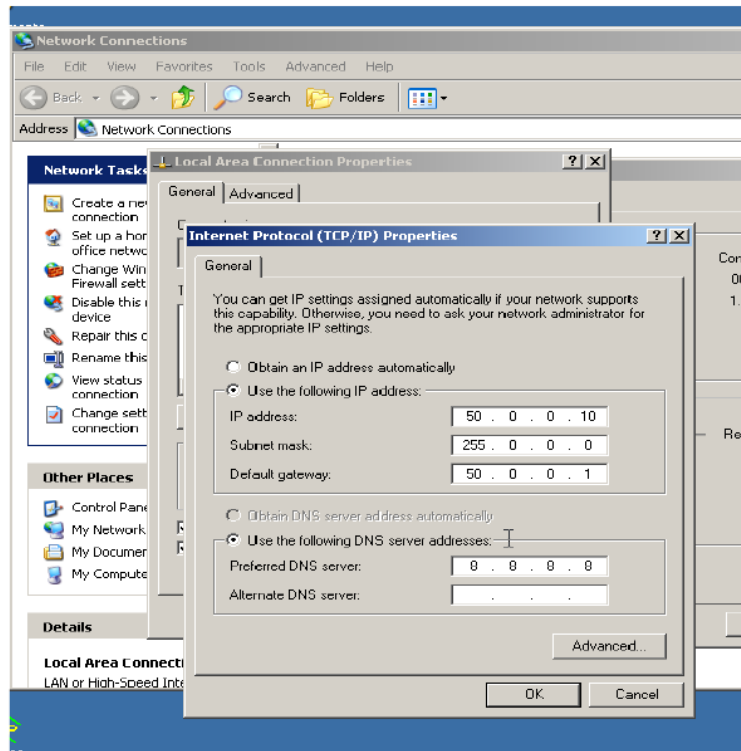


Figure 11 — Client PC configured to use 8.8.8.8 as preferred DNS server

TASK 12 Verifying the DNS from the Client PC

With DNS in place, both sites now resolve and load by domain name instead of by IP — confirming DNS and both web services work together end-to-end.

- <http://www.netflix.com> → resolves to 172.16.0.20 → IIS page loads
- <http://www.hotstar.com> → resolves to 172.16.0.10 → Linux/tthttpd page loads

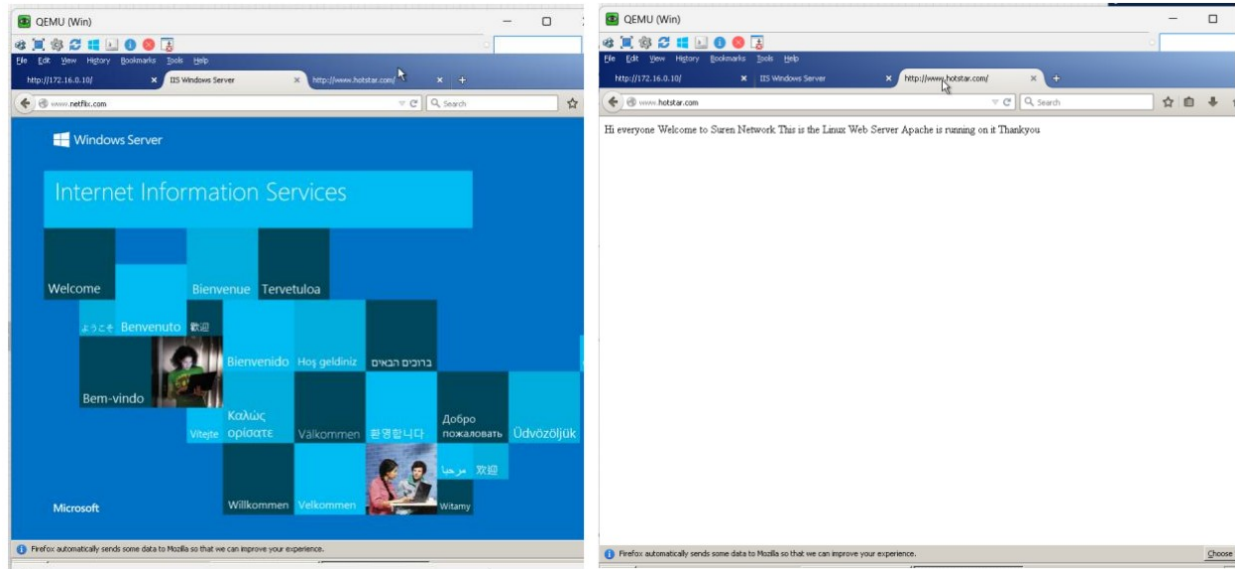


Figure 12 — Both www.netflix.com and www.hotstar.com loading by name, side by side

Conclusion

Every device in the topology — client PC, router, Windows Server, Linux server, and DNS server — was addressed and verified individually before layering services on top. IIS on the Windows Server and `thttpd` on the Linux server were each confirmed reachable by IP first, isolating routing from application issues. Once DNS records were added for both hosts and the client was pointed at the DNS server, both www.netflix.com and www.hotstar.com resolved correctly and served their respective pages — confirming the full stack (addressing → routing → web service → DNS) is working end-to-end.